Experiment : 6

Implementation of the Word Count Application Using Hadoop MapReduce

Mapping(CO)'s : $CO2$, $CO3$

Aim

To implement the Word Count application using Hadoop MapReduce and analyze the frequency of words in a text file.

Objectives

- To understand the MapReduce programming model.
- To write a Word Count program using Hadoop MapReduce.
- To compile and execute the MapReduce application.
- To store input and output data in HDFS.
- To analyze the generated word frequency results.

Prerequisites

- Basic knowledge of Java programming.
- Basic knowledge of Linux commands.
- Hadoop installed and configured.
- Java Development Kit (JDK).
- Eclipse IDE or Visual Studio Code (optional).
- HDFS running successfully.

Theory/Background

Apache Hadoop MapReduce is a programming framework used for processing large datasets in a distributed environment. It divides a large task into smaller subtasks and executes them in parallel across multiple nodes.

The **Word Count** application is one of the most common examples used to understand MapReduce.

The MapReduce model consists of two main phases:

- **Mapper:** Reads the input data and generates intermediate key-value pairs. In the Word Count program, each word is emitted as the key with a value of **1**.
- **Reducer:** Receives all values associated with the same key and sums them to produce the total count for each word.

The Word Count application demonstrates how Hadoop processes text files efficiently using distributed computing.

Algorithm/Methodology

- Install and configure Hadoop.
- Create a text input file.
- Upload the input file to HDFS.
- Write the Word Count MapReduce program in Java.
- Compile the Java program and create a JAR file.
- Execute the MapReduce job.
- Store the output in HDFS.
- Display the output file containing word frequencies.

Dataset Description

Dataset Used: Sample Text File

File Name: input.txt

Sample Data:

Hadoop is a big data framework.

Hadoop uses MapReduce.

MapReduce processes big data efficiently.

The input file contains multiple lines of text. The Word Count application counts the number of occurrences of each word.

Program/Implementation

The implementation includes:

- Mapper Class
- Reducer Class
- Driver Class
- Compilation and JAR creation
- Execution of the MapReduce job

Test Cases

Test Case	**Expected Result**
Upload input file to HDFS	File uploaded successfully
Execute MapReduce job	Job completes successfully

View output	Word frequencies displayed
Count repeated words	Correct frequency generated

Experimental Results

Observation

- The input text file was uploaded successfully to HDFS.
- The Word Count MapReduce program executed without errors.
- The Mapper generated intermediate key-value pairs.
- The Reducer calculated the total occurrence of each word.
- The output file stored the frequency of each word in HDFS.

Result

The Word Count application was successfully implemented using Hadoop MapReduce. The program correctly counted the occurrences of each word in the input file and stored the output in HDFS.

Conclusion

The experiment demonstrated the implementation of the Word Count application using the Hadoop MapReduce framework. The Mapper and Reducer worked together to process the input data and generate the word frequency count. This experiment provided practical knowledge of distributed data processing using Hadoop.

Exercises/Assignments

1. Modify the input file and observe the changes in the output.
2. Ignore common words such as **is**, **the**, and **a** in the Word Count program.
3. Execute the program with another text file.
4. Compare the output before and after modifying the input file.
5. Explain the role of the Mapper and Reducer in the Word Count application.